

235. Lowest Common Ancestor of a Binary Search Tree

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {  
    // Base case  
    if (root == null || root == p || root == q) {  
        return root;  
    }  
  
    // Search left and right  
    TreeNode left = lowestCommonAncestor(root.left, p, q);  
    TreeNode right = lowestCommonAncestor(root.right, p, q);  
  
    // If both sides found, root is LCA  
    if (left != null && right != null) {  
        return root;  
    }  
  
    // Otherwise return non-null child  
    return (left != null) ? left : right;  
}  
}
```

235. Lowest Common Ancestor of a Binary Search Tree

```
class Solution {  
  
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {  
  
        while (root != null) {
```

```

        if (p.val < root.val && q.val < root.val) {
            root = root.left; // go left
        }
        else if (p.val > root.val && q.val > root.val) {
            root = root.right; // go right
        }
        else {
            return root; // split point → LCA
        }
    }

    return null;
}
}

```

230. Kth Smallest Element in a BST

```

class Solution {

    int count = 0;
    int result = 0;

    public int kthSmallest(TreeNode root, int k) {
        inorder(root, k);
        return result;
    }
}

```

```

private void inorder(TreeNode node, int k) {
    if (node == null) return;

    // Left
    inorder(node.left, k);

    // Root
    count++;
    if (count == k) {
        result = node.val;
        return;
    }

    // Right
    inorder(node.right, k);
}

```

215. Kth Largest Element in an Array

```

import java.util.*;

class Solution {
    public int findKthLargest(int[] nums, int k) {

        PriorityQueue<Integer> minHeap = new PriorityQueue<>();

        for (int num : nums) {
            minHeap.add(num);

```

```

        if (minHeap.size() > k) {
            minHeap.poll(); // remove smallest
        }
    }

    return minHeap.peek();
}

```

347. Top K Frequent Elements

```

import java.util.*;

class Solution {

    public int[] topKFrequent(int[] nums, int k) {

        // Step 1: Frequency map
        Map<Integer, Integer> map = new HashMap<>();
        for (int num : nums) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        // Step 2: Min Heap based on frequency
        PriorityQueue<Integer> minHeap =
            new PriorityQueue<>((a, b) -> map.get(a) - map.get(b));
    }
}

```

```

        for (int num : map.keySet()) {
            minHeap.add(num);

            if (minHeap.size() > k) {
                minHeap.poll();
            }
        }

        // Step 3: Build result
        int[] result = new int[k];
        for (int i = k - 1; i >= 0; i--) {
            result[i] = minHeap.poll();
        }

        return result;
    }
}

```

373. Find K Pairs with Smallest Sums

```

import java.util.*;

class Solution {

    public List<List<Integer>> kSmallestPairs(int[] nums1, int[] nums2, int k) {

```

```
List<List<Integer>> result = new ArrayList<>();
```

```
if (nums1.length == 0 || nums2.length == 0 || k == 0) {  
    return result;  
}
```

```
// Min Heap → stores {index in nums1, index in nums2}
```

```
PriorityQueue<int[]> minHeap = new PriorityQueue<>(  
    (a, b) -> (nums1[a[0]] + nums2[a[1]]) - (nums1[b[0]] + nums2[b[1]]))  
);
```

```
// Initialize heap with (i, 0)
```

```
for (int i = 0; i < nums1.length && i < k; i++) {  
    minHeap.add(new int[]{i, 0});  
}
```

```
while (k-- > 0 && !minHeap.isEmpty()) {
```

```
    int[] curr = minHeap.poll();
```

```
    int i = curr[0];
```

```
    int j = curr[1];
```

```
    result.add(Arrays.asList(nums1[i], nums2[j]));
```

```
// Move to next element in nums2
```

```
if (j + 1 < nums2.length) {
```

```
    minHeap.add(new int[]{i, j + 1});
```

```
}
```

```
}
```

```
    return result;
```

```
}
```

```
}
```